

Network Science Data II

PHYS 7332, Fall 2025

Luca Santagata
Assignment 2

1 *Question 1 (15 points). More databases.*

There are lots of databases out there that are very different from SQL. Although SQL is probably what you'll see most often out in the real world, it isn't optimal for all use cases. Let's read up on a few other kinds of databases to better understand what's out there. For each of the databases listed below, please explain how it structures its data (e.g., key/value, tabular, etc.) and tell us what kinds of data/applications it's most useful for. Additionally, give us an example of a bad use case for that specific database and explain why it is poorly suited for that particular use case.

(a) Redis

- Redis is an in-memory key-value database that stores data as pairs of unique keys and their associated values. It supports several value types such as strings, lists, sets, sorted sets, and hashes, which makes it extremely fast for operations that require frequent reads and writes. Redis is most useful for caching, session management, leaderboards, real-time analytics, and message brokering, since all data is stored in memory and can be accessed in microseconds. However, Redis is poorly suited for applications that require complex relational queries or long-term persistent storage, such as accounting systems or data warehouses, because data persistence is optional and its memory-based design makes it expensive and inefficient for large datasets that do not fit into RAM.

(b) MongoDB

- MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like documents known as BSON. Each document can have a different structure, allowing for schema-less design and hierarchical nesting of objects. It is best suited for applications dealing with semi-structured data or rapidly evolving schemas, such as content management systems, product catalogs, or social media platforms. However, MongoDB performs poorly in use cases that require multi-table joins or complex transactions because it lacks strong ACID guarantees across multiple documents and can lead to data inconsistencies in highly relational workloads.

(c) Cassandra

- Cassandra is a wide-column NoSQL database that organizes data into tables with rows and columns but without a fixed schema, distributed across many nodes for high availability and scalability. Each row can have a different set of columns, and data is partitioned using a primary key to achieve fault tolerance. Cassandra is ideal for applications requiring massive scalability and continuous uptime, such as IoT data collection, time-series data, or large-scale messaging systems. Conversely, it is a poor choice for applications needing complex query capabilities or ad hoc analytics because it lacks support for joins, aggregation, and flexible querying, favoring instead denormalized, query-specific data models.

(d) Neo4j

- Neo4j is a graph database that structures data as nodes (entities), relationships (edges), and properties. It uses the Cypher query language to traverse relationships efficiently, making it ideal for highly connected data such as social networks, recommendation engines, fraud detection, and knowledge graphs. However, Neo4j is poorly suited for applications requiring large-scale transactional updates or simple key-based lookups (e.g., high-frequency trading systems), since its graph traversal operations are computationally expensive for flat or unconnected datasets, and its performance decreases when relationships are not central to the data model.

2 *Question 2 (40 points). Exploring community detection.*

- a) Download a network dataset from <https://networkrepository.com/> with between $1,000 < N < 5,000$ nodes (select a different dataset from the one you used in Assignment 1). Select a modularity maximization algorithm to detect communities in this graph.
- For this exercise I have used the [inf-openflights](#) network with 2,939 nodes and 15,677 edges.
- (i) Visualize your network, coloring the nodes by community membership.
- The visualization of the network communities found using [Louvain algorithm](#) is shown in [Figure 1](#).
- (ii) Report the modularity of the partition your algorithm found.
- The modularity value obtained from Louvain algorithm is 0.6540.
- (iii) Report the number of communities detected.
- The Louvain algorithm found 34 communities.
- (iv) Store your partition as a dictionary, in the form of `{node_id:community_id}`.

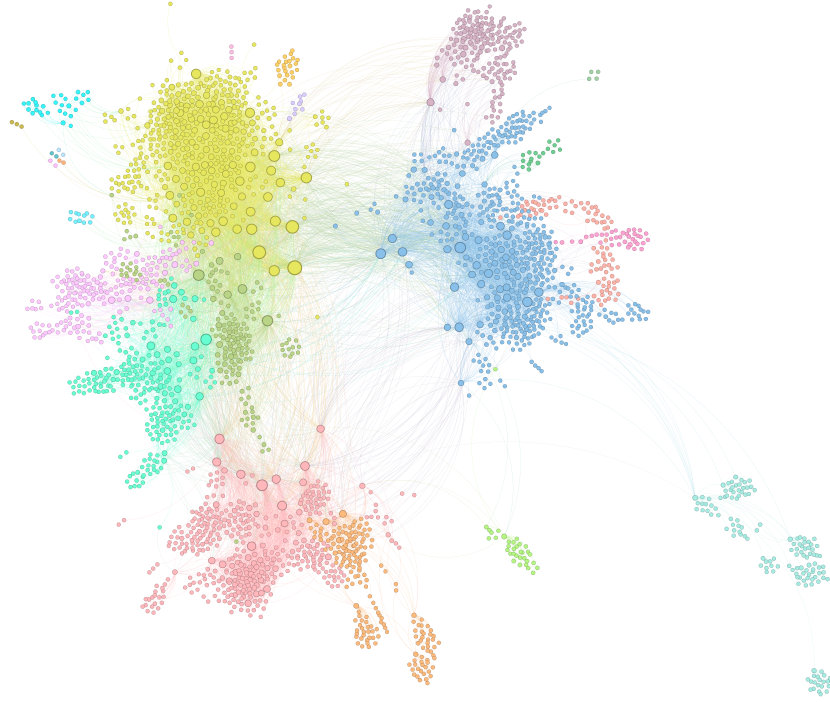


Figure 1: Communities visualization in the inf-openflights network detected by the Louvain algorithm.

b) Using the same network as in (a), use **graph-tool** to create a partition of your network (i.e., run community detection). Visualize your network again, coloring the nodes by community membership; use the same network visualization layout as the one in (a). Store your partition as a dictionary, in the form of `{node_id:community_id}`.

- To create the network partition, this time graph-tool's **Stochastic Block Model (SBM)** inference method was used, and the visualization with the same layout is shown in **Figure 2**. The resulting modularity obtained with graph-tool is 0.3597, corresponding to a total of 36 detected communities.

As shown in **Figure 3**, the two community detection methods, Louvain and graph-tool's Stochastic Block Model (SBM), rely on fundamentally different principles and therefore produce distinct results. The Louvain algorithm explicitly maximizes modularity, aiming to find groups of nodes that are densely connected internally and sparsely connected externally. In this case, Louvain identified 34 communities with a modularity value of 0.6540, indicating a highly modular structure. Conversely, the SBM implemented in graph-tool does not optimize modularity; instead, it infers the community structure by minimizing the description length (MDL) of a generative statistical model, following a Bayesian approach. This leads to a partition that balances model complexity and goodness of fit. The SBM identified 36 communities with a modularity value of 0.3597. The lower modularity does not imply a worse result but rather reflects the fact that the SBM

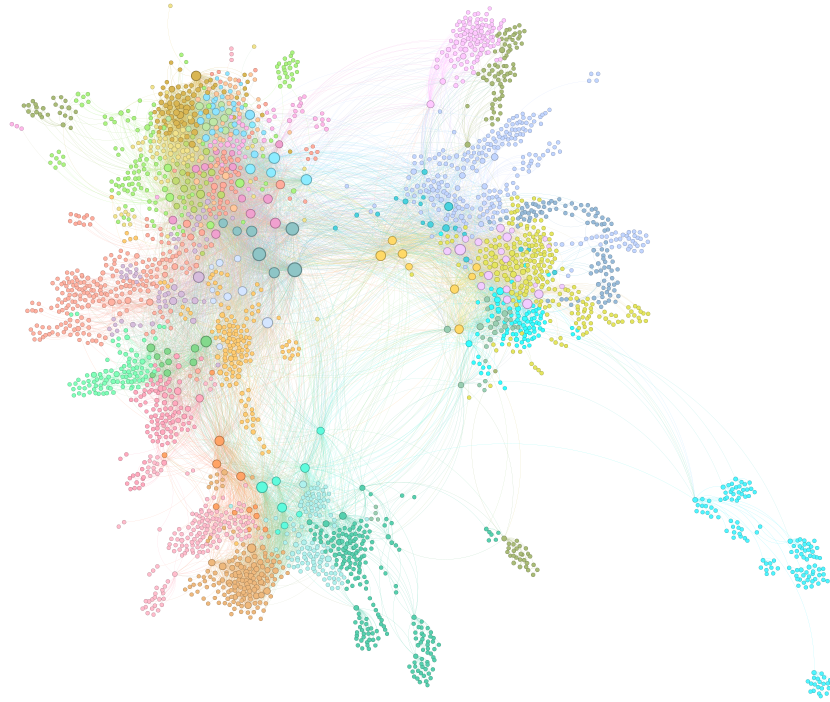


Figure 2: Communities visualization in the inf-openflights network detected by the Stochastic Block Model (SBM) inference method.

optimizes a different criterion: statistical plausibility rather than structural separation. In summary, Louvain emphasizes modularity maximization and tends to highlight clear-cut, dense clusters, while the SBM approach seeks the most probable underlying generative model of the network, offering a more statistically grounded description of its community organization.

- c) Create a degree-preserving randomization of your network, and run the two community detection algorithms from above on your randomized graph. Store each partition as a dictionary, in the form of `{node_id:community_id}`.
 - d) You should now have four different dictionaries, corresponding to the partitions created in (a–c). Create a figure with four subplots: in each, plot the community size distribution of each partition. Your figure should be publication-ready quality, with well-labeled, consistent axes, log-scaled where relevant, etc.
- [Figure 4](#) shows the community size distributions obtained with the Louvain and graph-tool’s Stochastic Block Model (SBM) methods for both the original and the randomized versions of the INF OpenFlights network. While both algorithms identify multiple communities in the original graph, only the SBM correctly detects a single community when applied to the randomized network. This behavior reflects the expected result: in the absence of real structural organization, a proper statistical model such as the SBM should identify one homogeneous

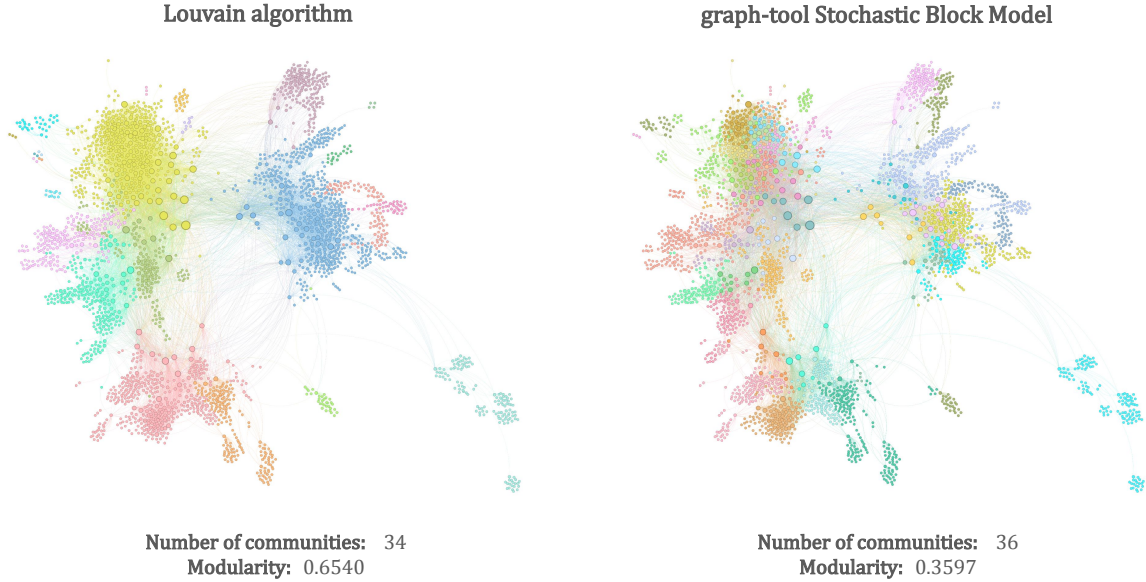


Figure 3: Comparison between the community structures obtained with the Louvain algorithm and graph-tool’s Stochastic Block Model (SBM) inference method.

group, whereas modularity-based methods like Louvain may still artificially partition random graphs due to modularity optimization biases.

- e) For the different partitions computed above, discuss how you would approach comparing these partitions quantitatively. For example, imagine a function $f(b_1, b_2)$ that takes two partitions (b_1 and b_2) as input and returns a number corresponding to how similar/dissimilar the partitions are. What properties should this function f have? For this question, base your answer on a technique used in the literature, after a brief search about methods for quantifying differences between graph partitions.
- To compare different network partitions quantitatively, we define a function $f(b_1, b_2)$ that measures the similarity or dissimilarity between two partitions b_1 and b_2 of the same set of nodes. Several methods have been proposed in the literature for this purpose, each with different properties and theoretical foundations. One of the most widely used approaches is the *Normalized Mutual Information (NMI)* introduced in the context of community detection by [Danon et al. \[2005\]](#) and later adopted by [Lancichinetti et al. \[2009\]](#). NMI is rooted in information theory and quantifies the amount of shared information between two partitions. Formally, for two partitions b_1 and b_2 , it is defined as:

$$\text{NMI}(b_1, b_2) = \frac{2 I(b_1, b_2)}{H(b_1) + H(b_2)} \quad (1)$$

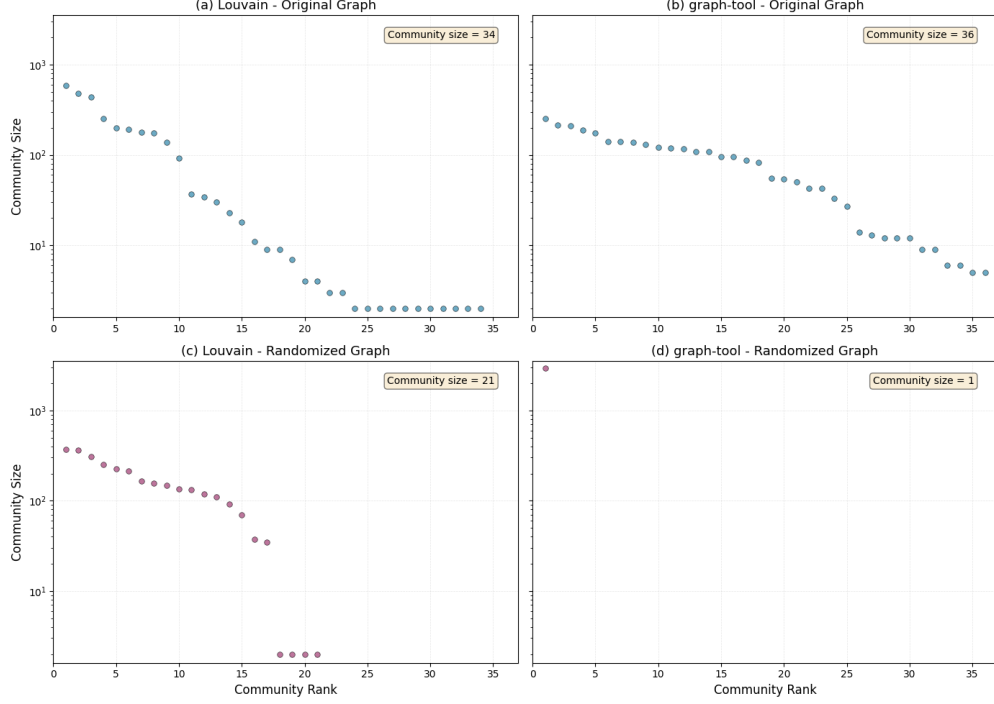


Figure 4: Community size distributions obtained with the Louvain and graph-tool’s Stochastic Block Model (SBM) methods for both the original and the randomized versions of the inf-openflights network.

where $I(b_1, b_2)$ is the mutual information between b_1 and b_2 , and $H(b_1)$ and $H(b_2)$ are their respective entropies:

$$I(b_1, b_2) = \sum_{i=1}^{K_1} \sum_{j=1}^{K_2} P(i, j) \log \frac{P(i, j)}{P(i)P(j)} \quad (2)$$

$$H(b_1) = - \sum_{i=1}^{K_1} P(i) \log P(i), \quad H(b_2) = - \sum_{j=1}^{K_2} P(j) \log P(j) \quad (3)$$

Here, $P(i)$ and $P(j)$ denote the probabilities that a randomly chosen node belongs to community i in b_1 and to community j in b_2 , respectively, while $P(i, j)$ is the joint probability that a node belongs to both community i in b_1 and community j in b_2 . NMI ranges between 0 and 1, where 1 indicates identical partitions and 0 indicates statistically independent ones.

An effective partition comparison function $f(b_1, b_2)$ should satisfy several key properties [Meilă \[2007\]](#):

- *Normalization*: $f(b_1, b_2)$ should return values within a fixed range (typically $[0, 1]$), enabling consistent comparisons across networks of different sizes.
- *Symmetry*: $f(b_1, b_2) = f(b_2, b_1)$, since the similarity should not depend on the order of the partitions.

- *Boundary conditions*: $f(b_1, b_2)$ should reach its maximum value when $b_1 = b_2$ (perfect agreement) and its minimum when the partitions are completely independent.
- *Label invariance*: The measure must be independent of the numerical labeling of communities, depending only on node groupings.
- *Sensitivity to merges and splits*: The measure should penalize both overpartitioning (splitting) and underpartitioning (merging) of communities.

Other widely used measures include:

- *Adjusted Rand Index (ARI)* Hubert and Arabie [1985]: Based on pair counting, ARI corrects for chance agreement and ranges from -1 (anti-correlation) to 1 (perfect match). It measures how many pairs of nodes are consistently placed in the same or different communities in both partitions.
- *Variation of Information (VI)* Meilă [2007]: A true metric derived from information theory, defined as:

$$VI(b_1, b_2) = H(b_1) + H(b_2) - 2I(b_1, b_2), \quad (4)$$

where lower values indicate higher similarity. Unlike NMI, VI satisfies the triangle inequality and can thus be used to cluster or average multiple partitions.

- *Modularity difference* Newman [2006]: When both partitions are obtained via modularity optimization, one can compare their modularity scores $Q(b_1)$ and $Q(b_2)$ directly:

$$\Delta Q = Q(b_1) - Q(b_2), \quad (5)$$

although this captures only the quality of each partition, not their structural similarity.

In practice, NMI and ARI are often preferred for community detection evaluation because they assess the structural correspondence between partitions rather than only comparing their modularity values. As discussed by Fortunato [2010], modularity-based comparisons can be misleading when partitions have similar Q values but substantially different community structures.

3 Question 3 (30 points). Benchmarks for community detection.

There are many ways to “benchmark” community detection algorithms; one famous approach is known as the Lancichinetti–Fortunato–Radicchi (LFR) benchmark¹.

- Read the LFR Benchmark paper (linked in the footnote above). Describe the core motivation behind the paper, its main contributions, and the basics of the benchmark (i.e., what are the inputs to the benchmark, what is the output, etc.).

¹Lancichinetti, A., Fortunato, S., & Radicchi, F. (2008). *Benchmark graphs for testing community detection algorithms*. Physical Review E, 78(4), 046110. <https://doi.org/10.1103/PhysRevE.78.046110>

- This paper addresses a crucial limitation in how community detection algorithms were being evaluated. Prior to this work, the standard for testing such algorithms was the Girvan–Newman (GN) benchmark, which assumed that all nodes had roughly the same degree and all communities had equal size. However, real-world networks display strong heterogeneity: both node degrees and community sizes typically follow power-law distributions. The authors argue that this discrepancy makes the GN benchmark an unrealistic and overly simplistic test for modern algorithms. Therefore, the main motivation of the paper is to introduce a more realistic, flexible, and challenging benchmark that better reflects the structural complexity of real networks and can more accurately reveal the strengths and weaknesses of community detection algorithms. The paper introduces a new class of artificial graphs that extends the GN model by incorporating key empirical properties of real networks. It defines both the node degree distribution and the community size distribution as power laws, with adjustable exponents γ and β , respectively, and introduces a *mixing parameter* μ , which controls the fraction of links that each node shares with nodes inside versus outside its community: a smaller μ indicates well-separated communities, while larger μ values produce fuzzier ones. The authors provide an efficient construction algorithm based on the configuration model, capable of generating large-scale networks (up to 10^5 – 10^6 nodes) in linear time with respect to the number of edges. The benchmark is then used to systematically test two popular community detection methods, modularity optimization and the Potts model clustering, showing that the new benchmark exposes limitations, such as the well-known *resolution limit* of modularity, that are not visible when using the simpler GN test. The LFR benchmark generates synthetic networks that possess both a built-in community structure and realistic heterogeneity in node degrees and community sizes. The inputs of the benchmark are the number of nodes N , the average degree $\langle k \rangle$, the degree distribution exponent γ , the community size exponent β , the mixing parameter μ , and the ranges for node degrees ($k_{\min}$, $k_{\max}$) and community sizes ($s_{\min}$, $s_{\max}$). The output is a synthetic network that includes a degree sequence following a power-law distribution, communities of heterogeneous sizes, edges assigned according to the desired level of inter-community mixing μ , and the ground-truth community labels for each node, which allow quantitative evaluation of algorithms (e.g., via *Normalized Mutual Information*, NMI). In summary, the paper introduces the LFR benchmark as a realistic, tunable, and scalable testbed for community detection. By combining heterogeneous degree and community size distributions with a controllable level of mixing, it provides a much more stringent and informative framework for evaluating algorithms than earlier benchmarks. The results of the paper demonstrate that this new benchmark can reveal algorithmic limitations, such as the modularity resolution limit and size-dependent performance degradation, that remain hidden under simpler testing conditions, making it a foundational contribution to the study and validation of community detection methods.

(b) On page 2 of the article—*Section II: The Benchmark*—the authors describe the five

steps involved in sampling benchmark graphs:

- i. Step 1 is reminiscent of the procedure for creating power-law configuration model networks. Write a function that outputs a power-law degree sequence, $\{k_1, k_2, \dots, k_n\}$, which can be used as input in a configuration model. Your function should take n (network size), γ (power-law degree exponent), and $\langle k \rangle$ (average degree) as inputs.
 - The implementation that generates a power-law degree sequence can be found [here](#). The function has been tested for several parameter combinations, and the resulting empirical degree distributions are shown in [Figure 5](#). Each panel compares the sampled empirical distribution (blue points) with the theoretical scaling $P(k) \propto k^{-\gamma}$ (red line). The results confirm that the implemented sampling procedure successfully reproduces the expected power-law behavior across different exponents and target average degrees.
 - ii. Step 2 introduces a mixing parameter, μ , that is used to assign a fraction of a node's total degree, k_i , to be *internal* community edges (i.e., connecting to nodes with the same community label as node i), such that $k_i^{\text{in}} = k_i(1 - \mu)$. With probability μ , the remaining edges of i are connected to nodes *external* to the community, $k_i^{\text{out}} = k_i\mu = k_i - k_i^{\text{in}}$. Write a function that implements this step.
 - The implementation can be found [here](#).
 - iii. Step 3 resembles Step 1, but instead of defining power-law degree values, it defines a power-law *community size* distribution. What additional considerations does the paper say are needed in order to implement this step? No code is required to answer this.
 - In Step 3, the paper explains that the sizes of the communities are drawn from a power-law distribution with exponent β , analogously to how node degrees are generated in Step 1. However, several additional constraints are required to properly implement this step. Specifically, the total sum of all community sizes must equal the total number of nodes N in the network, ensuring that every node is assigned to some community. Moreover, the minimum and maximum community sizes, $s_{\min}$ and $s_{\max}$, must be chosen such that $s_{\min} > k_{\min}$ and $s_{\max} > k_{\max}$. These constraints guarantee that nodes of any degree can be accommodated within at least one community.
 - iv. *(Optional) Finish writing the rest of the function, putting Steps 1–5 together. Note: This is a challenging function to implement and, as such, is optional. If you don't have time to implement this, feel free to explore existing (proper) implementations of this algorithm, either online or with AI assistance.*
- (c) The `networkx` library has a built-in function, `nx.LFR_benchmark_graph()`, that can generate these benchmark graphs. Many users of this algorithm report its inconsistency in convergence. Explore this function's usage for yourself: Are there certain parameter combinations that make the algorithm converge more or less successfully? Based on your reading of the original LFR paper and your experiments above, briefly describe why you think this function has difficulty converging.

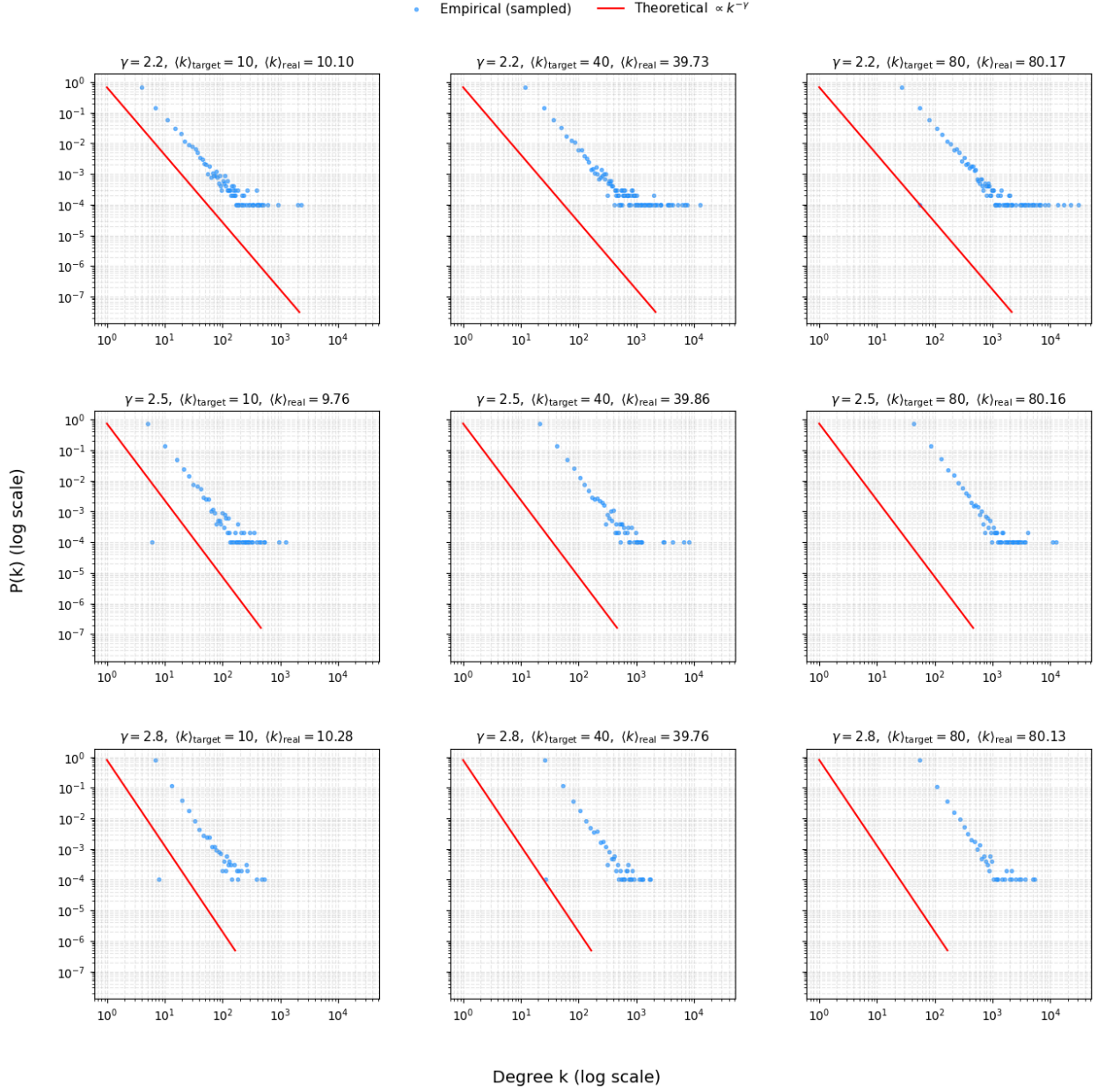


Figure 5: Empirical vs theoretical degree distributions for sampled power-law sequences.

- In my [experiments](#) with `nx.LFR_benchmark_graph()`, I found that convergence is highly sensitive to the chosen parameter combinations. The generator frequently failed with the error `ExceededMaxIterations: Could not match average_degree` when targeting high density (for instance, `average_degree` ≥ 30), when community caps were tight (e.g., `max_community` = `max_degree`), or when the mixing parameter μ was large (typically $\mu \geq 0.5$). These parameter regions create conflicting constraints: each node must realize an internal degree $(1 - \mu)k$ and an external degree μk , while the graph as a whole must simultaneously satisfy both a power-law degree distribution and a power-law community-size distribution under the

specified bounds. When the desired `average_degree` is too high for the available `min_community` and `max_community`, or when communities are too small relative to node degrees, the feasible space for constructing a valid graphical realization becomes extremely limited, and the algorithm exhausts its iteration budget without finding a consistent solution. By contrast, parameter settings with smaller μ , lower `average_degree`, and looser community caps converged reliably across multiple random seeds. This behavior is consistent with the observations reported in [Lancichinetti et al. \[2008\]](#), which emphasizes that the feasibility of the benchmark depends on the joint satisfaction of three interdependent constraints—the degree sequence, the community-size distribution, and the per-node mixing fraction. When these constraints approach the boundary of mutual incompatibility (for instance, in dense, highly mixed, or tightly capped community regimes), the iterative construction method cannot converge, leading to the inconsistent behavior often observed in the NetworkX implementation.

4 *Question 4 (15 points). Python packages and software management.*

Sometimes, you want to put a chunk (or several related chunks) of Python code into a package that you or others can install. This helps you and others use your code across several different projects and organize it in ways that make sense for you. You’ve developed several useful functions in this class so far; now it’s time to turn them into a package for your future use. You may find [this tutorial](#) and its sequel, on Python packaging and making a package installable, to be useful.

- (i) Distill some of the code we’ve written in this class so far into a basic toolkit that you’ll reuse. Pick 6–10 functions (or objects) that you think would make up a useful network science toolkit based on code we’ve worked on in class. **Note:** You should not be planning on writing much new code for this question!
- (ii) Create a new GitHub repository with a `README.md` file that thoroughly explains the functions and/or objects you put in this package. Link it in your answer.
- (iii) Following the tutorials linked above, copy your code into a file(s) in your new repository, making sure it’s organized in a way that works for you.
- (iv) Again, following the tutorials linked above, make your repository/package pip installable. If everything works correctly, we should be able to clone your repository, `pip install` it into an environment, and import your functions/objects.
 - the information about the Python package and the instructions on how to install and use it can be found [here](#).

References

- Leon Danon, Albert Diaz-Guilera, Jordi Duch, and Alex Arenas. Comparing community structure identification. *Journal of statistical mechanics: Theory and experiment*, 2005 (09):P09008, 2005.
- Santo Fortunato. Community detection in graphs. *Physics reports*, 486(3-5):75–174, 2010.
- Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of classification*, 2(1):193–218, 1985.
- Andrea Lancichinetti, Santo Fortunato, and Filippo Radicchi. Benchmark graphs for testing community detection algorithms. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, 78(4):046110, 2008.
- Andrea Lancichinetti, Santo Fortunato, and János Kertész. Detecting the overlapping and hierarchical community structure in complex networks. *New journal of physics*, 11(3):033015, 2009.
- Marina Meilă. Comparing clusterings—an information based distance. *Journal of multivariate analysis*, 98(5):873–895, 2007.
- Mark EJ Newman. Modularity and community structure in networks. *Proceedings of the national academy of sciences*, 103(23):8577–8582, 2006.